

Assignment #3:-

Question # 1:-

Dense prediction is a type of machine learning task where the model produces an output for everything pixel in an image, rather than a single output for the whole image.

Difference

Image Classification	Object Detection	Dense Prediction
Assigns one single label to the entire image (e.g. "This image is a dog").	Identifies objects and draws bounding boxes around them, outputting coordinates and labels (e.g. "Dog at coordinates X, Y")	Assign a label to every pixel (e.g., "Pixel 1-100 are dog, 101-200 are grass")

Real World Example: Autonomous driving, where semantic segmentation is essential to classify every pixel as road, pedestrian, vehicle, or sidewalk so the car can navigate safely.

Question # 2 :-

Why reduction happens :

Standard CNNs use pooling layers (like Max Pooling) and convolutions with slides greater than 1 to summarize local features, which inherently shrinks the spatial dimensions (width and height) of the feature maps.

Problem for segmentation:

Dense tasks require pixel-perfect outputs that match the original image resolution. If the resolution is drastically reduced, the model cannot make predictions for every original pixel.

Information lost: Fine-grained spatial information, exact edge locations and precise localization data are lost.

Question # 3 :-

Compute final feature map size

Formula for 2×2 pooling with stride

$$2: \text{Output} = \frac{\text{Input}}{2}$$

Start : 256×256

After Pool 1 : 128×128

After Pool 2 : 64×64

After Pool 3 : 32×32

Final Size : 32×32

Percentage of spatial resolution retained

Original pixels : $256 \times 256 = 65,536$

Final pixels : $32 \times 32 = 1,024$

Retained : $\frac{1024}{65536} \times 100 = 1.5625\%$

Impact :

Nearly 98.4 % of spatial detail is discarded. This makes it impossible to directly map the deep features back to precise, pixel-level boundaries without unsampling.

Question # 4:-

Conversion : A classification CNN is converted into an FCN by replacing the final fully connected (FC) layers with Convolutional layers.

What happens to FC layers: An FC layer

that expects a specific flat input size is replaced by a 1×1 or $N \times N$ convolutional layer that spans the entire spatial dimension of the preceding feature map.

Variable input size: FC layers require fixed input vectors (meaning fixed image sizes). Convolutional filters slide over inputs of any spatial size, so an FCN can process variable sized images and simply output proportionally sized feature maps.

Question # 5:-

Equivalent Convolutional layer: You replace the FC layer with a standard 2D convolutional layer.

Filter size and number of filters: To match the FC layer's global view of the 7×7 input, the filter size must be exactly 7×7 . Because the original FC layer has 4096 neurons, you need 4096 filters.

Output Dimension: Applying 4096 filters of

size 7×7 to a 7×7 spatial input
(with no padding) results in a spatial
size of 1×1 . The output dimension is
 $1 \times 1 \times 4096$

Question no 6:-

Nearest neighbor: Copies the value of the
nearest pixel to fill new space

- Learnability: No Accuracy: ~
- Accuracy: Low / Blocky
- Complexity: Very low

Transposed Convolution: Uses a set of
learnable weights to 'broadcast' a single
input value into a larger output field

- Learnability: Yes
- Accuracy: High
- Complexity: High

Question no 7:-

Estimated output size:

Formula without output padding:

$$\text{Output} = (\text{Input} - 1) \times \text{Stride} + \text{kernel}$$

$$\text{Output} = (16 - 1) \times 2 + 4$$

$$= (15 \times 2) + 4$$

$$= 34 \times 34$$

(Note: with standard padding, this is usually designed to exactly double the input to 32×32)

Why it increases resolution: It increases resolution by effectively placing gaps (zeros) between the original input pixels and then sliding a convolutional kernel over this expanded space, allowing a single pixel to influence a larger patch of output pixels.

Question no 8:-

Why they are used: To recover the fine-grained spatial details that were lost during downsampling.

What is combined: Feature maps from the contracting / downsampling path (encoder) are concatenated or added to the corresponding feature maps in the expanding

/upsampling path (decoder).

Importance of early layers: Early layers contain high-resolution, low-level features (like edges, corners, and exact textures).

Localization vs. Semantic: Deep layers provide strong semantic understanding (they know 'what' the object is but not exactly "where" its borders are). Early layers provide strong localization (they know exactly "where" the borders are). Skip connections merge both.

Question no 9:-

Output tensor size: For a 128×128 image with 4 classes, the tensor is $128 \times 128 \times 4$.

What each pixel represents: The depth of 4 at each spatial location represents the unnormalized scores (logits) or probabilities for each of the 4 classes for the specific pixel.

Activation function: Softmax is applied pixel-wise along the channel / depth dimension. It ensures that for every individual pixel, the probabilities across the 4 classes sum up to 1.

Question no 10:-

Input layer: Takes in the road scene RGB image (e.g. $256 \times 256 \times 3$)

Downsampling: Passes the image through blocks of standard Convolutions, ReLUs, and Max Pooling layers. This extracts abstract features ('cars', 'road') while reducing spatial size (e.g. down to $16 \times 16 \times 512$)

Bottleneck: The deepest layer where spatial resolution is lowest but feature depth is highest. Represents the purest semantic 'meaning' of the image.

Upsampling (Decoder): Uses transposed convolutions to gradually increase the spatial size back towards 256×256

Skip connections : At each upsampling steps, the corresponding high-resolution feature map from the encoder is concatenated with the decoder's current map, combining 'what' with 'where'.

Final output layer: A 1×1 convolution layer reduces the channel depth to the number of classes (e.g. N), followed by a pixel-wise Softmax activation, yielding a dense prediction map.

Question no 11:-

Feedforward limitations: Traditional networks assume inputs are independent and required fixed-size inputs and outputs. They have no 'memory' to connect previous phase inputs to current ones.

How RNN solves it: RNNs use loops within their architecture. They maintain an internal 'hidden state' that gets updated at each step allowing information from past sequence elements to persist and influence future outputs.

Examples: Text processing (e.g. language translation)
and time series analysis (e.g. stock price forecasting)

Question no 12:-

Input (x_t) The data point at the current time step t .

hidden state (h_t): The "memory" vector at time step t , calculated by combining the current input (x_t) and previous hidden state (h_{t-1})

Output: The prediction at time step t , typically calculated directly from the current hidden state (h_t).

Role of Hidden State: It acts as the network's working memory, carrying contextual information forward from time step 0 up to time step t .

Question no 13:-

Given:

$$h_t = \tanh(W_h h_{t-1} + W_x x_t)$$

$$h_{t-1} = 0.5$$

$$x_t = 1$$

$$W_h = 0.8$$

$$W_x = 0.2$$

Input to activation:

$$\begin{aligned} &= (W_h \cdot h_{t-1}) + (W_x \cdot x_t) \\ &= (0.8 \times 0.5) + (0.2 \times 1) \\ &= 0.4 + 0.2 \\ &= 0.6 \end{aligned}$$

Compute h_t :

$$\begin{aligned} h_t &= \tanh(0.6) \\ &\approx 0.537 \end{aligned}$$

Representation: The value 0.537 is newly updated memory state for the current time step. It safely compresses both the past context (0.4) and the new input (0.2) into a normalized range between -1 and 1.

Question no 14:-

Meaning: Unrolling means visualizing the RNN loop as a flat sequence of individual neural network layers, one for each time step in the input sequence, where all layers share the exact same weights.

Importance for training: Neural networks are

trained by calculating gradients across layers. You cannot calculate a gradient through a 'loop'. By unrolling, we create a finite directional computational graph.

Relation to Backpropagation: Unrolling is strictly necessary to perform Backpropagation Through Time (BTT) as it allows the error to flow backward from the last time step through all previous time steps.

Question no 15:-

One - to - one : A single input produces a single output (Standard neural network). Example: Basic Image Classification.

One - to - many: A single input produces a sequence of outputs. Example: Image Captioning (Image $\rightarrow$ Sequence of words).

Many - to - one: A sequence of inputs produces a single output. Example: Sentiment Analysis (Paragraph of text $\rightarrow$ 'Positive / Negative').

Many - to - many: A sequence of inputs

produces a sequence of outputs. Example:
Machine Translation (English sentence $\rightarrow$ French sentence)

Question no 16 :-

What is it: BPTT is the standard backpropagation algorithm applied to the "unrolled" computational graph of an RNN.

Difference: Unlike standard backpropagation, BPTT must sum the gradients for the shared weights across every single time step.

The error at time t affects the weights at time t , $t-1$, $t-2$ etc.

Computationally Expensive: It requires storing the hidden states and inputs for every single time step during the forward pass in order to compute the chain rule backward. For long sequences, this requires immense memory.

Question no 17 :-

Explanation: In long sequences, error gradients for

late time steps must be multiplied by weight matrices repeatedly to pass back to earlier time steps.

Why it occurs: If the weights or the derivatives of the activation functions (like tanh or sigmoid) are less than 1, repeated multiplication cause the gradients to shrink exponentially toward zero.

Effect: The network completely stops learning long-term dependencies. The earliest inputs get no weight updates, so the model only remembers recent inputs.

Question no 18:-

What is it: The inverse of the vanishing gradient. If weights are large (> 1), repeated multiplication during BPTT causes gradients to grow exponentially large.

Effect: It leads to massive, unstable weight updates during training often causing the model's loss to spike or output NaN (Not a number) values.

Methods to Handle : Gradient Clipping, which forcibly caps the maximum value of gradients of a specific threshold before applying the weight update.

Question no 19:-

Why introduced: LSTMs were specifically designed to combat the vanishing gradient problem in standard RNNs.

Problem solved: They allow the network to effectively learn and remember long-term dependencies across hundred of time steps.

Memory Cell and Gates: Instead of constantly overwriting the hidden state, LSTMs maintain an internal "cell state" (a conveyor belt of memory). They use mathematically "gates" (sigmoid + pointwise multiplication) to optionally let information through an input gate (what new data to add), a forget gate (what old data to erase), and an output gate (what part of memory to output right now).

Question no 20 -

Type of Structure: Many-to-one RNN

Input Representation: The sentence is tokenized into words. Each word is mapped to a dense vector using an embedding layer (e.g. Word2Vec).

RNN Processing Sequence: The embedded words are fed into an LSTM sequence, one time step at a time. The LSTM updates its hidden state at each word, gradually building the context of the whole sentence.

Final Output Layer: After the final word is processed, we extract the final hidden state (which now summarizes the whole sentence). We pass this state through a fully connected (Dense) layer with a sigmoid activation function to output a probability between 0 and 1 (e.g. > 0.5 is positive, < 0.5 is negative).